

DESARROLLO WEB EN ENTORNO CLIENTE

6. UTILIZACIÓN DEL MODELO DE OBJETOS DEL
DOCUMENTO (DOM)

6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.1. EL MODELO DE OBJETOS DEL DOCUMENTO (DOM)

DOM:

- Estándar W3C que define cómo acceder a documentos (HTML, XML...).
- Interfaz de programación de aplicaciones (API) del W3C.
- Mediante scripts se puede acceder y actualizar el contenido dinámicamente.

Tipos:

DOM CORE:
modelo para
cualquier
documento
estructurado.

XML DOM:
modelo
estándar para
documentos
XML.

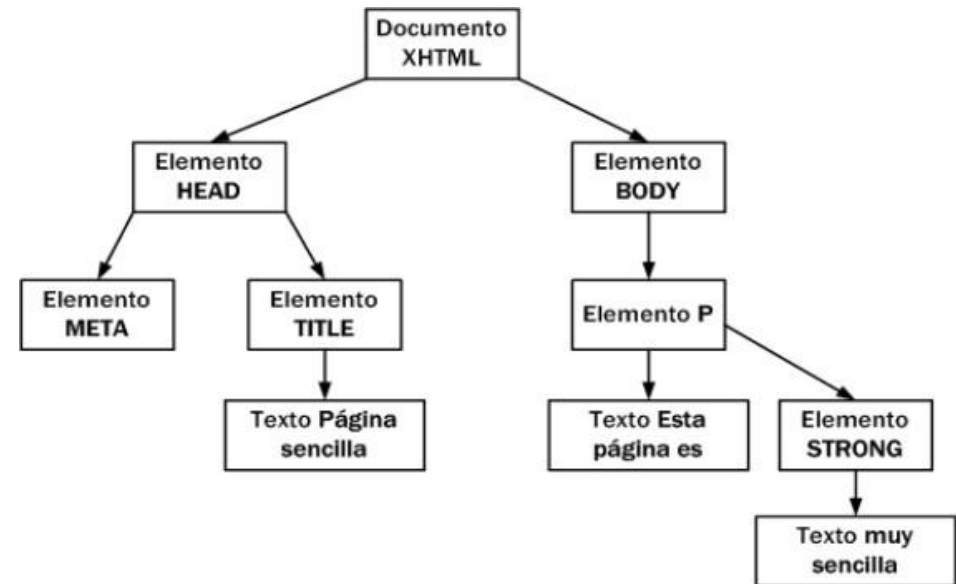
HTML DOM:
modelo estándar
para
documentos
HTML



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.1. EL MODELO DE OBJETOS DEL DOCUMENTO (DOM)

```
<!DOCTYPE html PUBLIC "
//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1
-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type"
      content="text/html; charset=iso-8859-1" />
<title>Página sencilla</title>
</head>
<body>
<p>Esta página es <strong>muy sencilla</strong></p>
</body>
</html>
```



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.1. EL MODELO DE OBJETOS DEL DOCUMENTO (DOM)

Ordenación de la estructura del árbol:

- En el árbol de nodos el superior es la raíz.
- Cada nodo, salvo la raíz, tiene un padre.
- Un nodo puede tener cualquier número de hijos.
- Una hoja es un nodo sin hijos.
- Nodos con el mismo padre son hermanos.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.2. OBJETOS DEL MODELO. PROPIEDADES Y MÉTODOS

Nodos más importantes

Elemento	Descripción
Document	Es el nodo raíz del documento HTML, todos los documentos del árbol cuelgan de él
DocumentType	Representación del DTD de la página. Un DTD es una definición de tipo de documento. Define la estructura y sintaxis de un documento XML. El DOCTYPE es el encargado de indicar el DocumentType.
Element	Representa el contenido de una pareja de etiquetas de apertura y cierre (<etiqueta>...</etiqueta>). También puede representar una etiqueta abreviada que se cierra a sí misma (). Este es el único nodo que puede tener tantos nodos hijos como atributos.
Attr	Representa el nombre de un atributo o valor.
Text	Almacena la información que es contenida dentro de un nodo Element.
CDataSection	Representa una secuencia de código de tipo <![CDATA[]]>. Este texto solamente será analizado por un programa de análisis.
Comment	Representa un comentario XML.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.2. OBJETOS DEL MODELO. PROPIEDADES Y MÉTODOS

Interfaz Node

- Javascript crea un objeto llamado Node para manipular la interfaz de los nodos.
- Define una serie de constantes que identifican los tipos de nodo (diapositiva siguiente).
- También proporciona propiedades y métodos.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.2. OBJETOS DEL MODELO. PROPIEDADES Y MÉTODOS

Constantes que identifican los tipos de nodo



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.2. OBJETOS DEL MODELO. PROPIEDADES Y MÉTODOS

Propiedades y métodos

Propiedad/Método	Valor devuelto	Descripción
nodeName	String	Nombre del nodo (no está definido para algunos tipos de nodo)
nodeValue	String	Valor del nodo (no está definido para algunos tipos de nodo). Ej. Para <code><p>Texto</p></code> , el <code>nodeValue</code> devolverá "Texto".
nodeType	Number	Una de las 12 constantes definidas anteriormente.
ownerDocument	Document	Referencia al documento al que pertenece el nodo.
firstChild	Node	Primer nodo de la lista de <code>childNodes</code> .
lastChild	Node	Último nodo de la lista de <code>childNodes</code> .
childNodes	NodeList	Lista de todos los hijos del nodo actual.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.2. OBJETOS DEL MODELO. PROPIEDADES Y MÉTODOS

Propiedades y métodos

Propiedad/Método	Valor devuelto	Descripción
previousSibling	Node	Referencia del hermano anterior o null si es el primer hermano.
nextSibling	Node	Referencia del hermano siguiente o null si es el último hermano.
childNodes()	Boolean	Devuelve true si el nodo actual tiene uno o más hijos.
attributes	NamedNode Map	Con nodos de tipo Element. Contiene objetos de tipo Attr que definen los atributos del elemento.
appendChild(nodo)	Node	Añade un nuevo nodo al final de la lista childNodes.
removeChild(nodo)	Node	Elimina un nodo de la lista childNodes.
replaceChild(nuevo,anterior)	Node	Reemplaza el nuevo nodo por el nodo anterior.
insertBefore(nuevo, anterior)	Node	Inserta un nuevo nodo antes de la posición del nodo anterior.

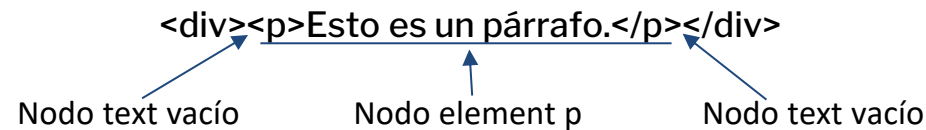


6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.2. OBJETOS DEL MODELO. PROPIEDADES Y MÉTODOS

Propiedades y métodos

Muchas de las anteriores propiedades presentan un problema, y es que devuelven también nodos de texto y de comentario.



Propiedad	Valor devuelto	Descripción
firstElementChild	Element	Devuelve el primer elemento hijo o null si no los tiene.
lastElementChild	Element	Devuelve el último elemento hijo o null si no los tiene.
previousElementSibling	Element	Devuelve el elemento hermano anterior o null si es el primero.
nextElementSibling	Element	Devuelve el elemento hermano anterior o null si es el último.
children	Array	Devuelve una colección HTML viva con todos los elementos hijo.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Ejemplo de código en HTML:

```
<html>
<head>
<title>TituloDOM</title>
</head>
<body>
<p>ParrafoDOM</p>
<p>ParrafoDOM segundo</p>
<p>ParrafoDOM tres</p>
</body>
</html>
```

Nodo raíz:

```
var obj_html= document.documentElement;
```

Primer y último hijo del nodo:

```
var obj_head= obj_html.firstChild;
```

```
var obj_body= obj_html.lastChild;
```

Acceso a nodos desde el índice de un array:

```
var obj_head= obj_html.childNodes[0];
```

```
var obj_body= obj_html.childNodes[1];
```

Acceso al número de hijos (longitud del array):

```
var numeroHijos= obj_html.childNodes.length;
```



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Acceso a los tipos de nodo (extrayendo el valor):

- `obj_tipo_document= document.nodeType; //9, o DOCUMENT_NODE`
- `obj_tipo_elemento= document.documentElement.nodeType; //1, o ELEMENT_NODE`

Acceso a los tipos de nodo (comparando con el valor):

- `alert(document.nodeType == Node.DOCUMENT_NODE); // true`
- `alert(document.documentElement.nodeType == Node.ELEMENT_NODE); // true`

Acceso al texto de un nodo de tipo texto:

- Extraer el texto de un nodo: `var x = document.getElementById("miNodo").textContent;`
- Cambiar el texto de un nodo: `document.getElementById("miNodo").textContent = "Paragraph changed!";`



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Acceso directo a los nodos:

getElementsByTagName()

- Recupera los elementos de la página HTML que le hayamos pasado por parámetro.
- `var divs= document.getElementsByTagName("div");`

getElementByName()

- Recupera los datos de la página HTML donde “*name*” coincide con el name que le hayamos pasado en la función.
- `var divPrimero= document.getElementByName("primero");`

getElementById()

- Recupera el elemento HTML cuyo ID coincida con el nombre pasado en la función. Ejemplo:
- `var elemento1 = document.getElementById("elemento1");`



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Acceso directo a los nodos:

getElementsByClassName()

- Devuelve una colección de elementos con un nombre de clase específico.
- `var divs= document.getElementsByClassName("miClase");`

querySelector()

- Devuelve el primer elemento del documento que coincida con el grupo especificado de selectores.
- `var divPrimero= document.querySelector("div");`
- `var divMiClasePrimero = document.querySelector("div.miClase");`

querySelectorAll()

- Devuelve una NodeList estática (no viva) que representa una lista de elementos del documento que coinciden con el grupo de selectores indicados.
- `var matches = document.querySelectorAll("div.nota, div.alerta");`

6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Acceso a los atributos de un nodo de tipo element:

`getNamelItem(nomAttr)`

- Devuelve el nodo de tipo attr cuya propiedad nodeName contenga el valor nomAttr.

`removeNamelItem(nomAttr)`

- Elimina el nodo de tipo attr en el que la propiedad nodeName coincida con el valor nomAttr.

`setNamelItem(nodo)`

- Añade el nodo attr a la lista de atributos del nodo element. Lo indexa según la propiedad nodeName del atributo.

`item(pos)`

- Devuelve el nodo correspondiente a la posición indicada por el valor numérico pos.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Acceso a los nodos de tipo atributo:

`getAttribute(nomAtributo)`

- Equivale a `attributes.getItem(nomAtributo)`

`setAttribute(nomAtributo, valorAtributo)`

- Equivale a `attributes.getNamedItem(nomAtributo).value=valor`

`removeAttribute(nomAtributo)`

- Equivale a `attributes.removeItem(nomAtributo).`



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

3. ACCESO AL DOCUMENTO DESDE CÓDIGO

EJERCICIO: u6e1_analisispagina:

- *Dispones del código de una página web en html: u6e1_analisispagina_texto.html.*
- *Introduce en el apartado de script el código necesario para extraer:*
 - *El número de párrafos de la página.*
 - *El texto del segundo párrafo.*
 - *El número de enlaces de la página.*
 - *La dirección del primer enlace.*
 - *La dirección del penúltimo enlace.*
 - *El número de enlaces que apuntan a /wiki/Balaenidae*
 - *El número de enlaces del primer párrafo.*
- *Para agregar texto en la página deberás introducir una etiqueta div con el id=info y añadir en ella toda la información detallada mediante:*
 - *Var info=document.getElementById("info");*
 - *Info.innerHTML = "Texto informativo";*



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Creación y eliminación de nodos

`createAttribute(nomAtributo)`

- Crea un nodo de tipo atributo con el nombre pasado a la función.

`createCDataSection(texto)`

- Crea una sección de tipo CDATA con un nodo hijo de tipo texto con el valor pasado.

`createComment(texto)`

- Crea un nodo de tipo comentario con el contenido del texto pasado.

`createDocumentFragment():`

- Crea un nodo de tipo DocumentFragment.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Creación y eliminación de nodos

`createElement(nomEtiqueta)`

- Crea un elemento del tipo etiqueta, del tipo del parámetro pasado como `nomEtiqueta`.

`createEntityReference(nomNodo)`

- Crea un nodo de tipo `EntityReference`.

`createProcessingInstruction(objetivo.dato)`

- Crea un nodo de tipo `ProcessingInstruction`.

`createTextNode(texto)`

- Crea un nodo de tipo texto con el valor del parámetro pasado.



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Ejemplo de creación de nodos en un documento:

```
var h = document.createElement("h1"); //crea el elemento h1  
var t = document.createTextNode("Hola, mundo"); //crea el nodo de texto  
h.appendChild(t); //añade el texto al elemento h1
```

```
var att = document.createAttribute("class"); //crea el atributo  
att.value = "prueba"; // añade el valor del atributo  
h.setAttributeNode(att); //añade el atributo al nodo h1 creado
```

RESULTADO: <h1 class="prueba">Hola, mundo</h1>



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

PARA AÑADIR UN ELEMENTO AL BODY:

```
document.body.appendChild(h);
```

RESULTADO:

```
<body>
  <h1
    class="prueba">Hol
a, mundo</h1>
</body>
```

PARA AÑADIR UN ELEMENTO A OTRO ELEMENTO QUE YA ESTÁ EN EL HTML::

```
var miDiv = document.getElementById("miDiv");
miDiv.appendChild(h);
```

RESULTADO:

```
<body>
  <div id="miDiv">
    <h1
      class="prueba">Hola,
mundo</h1>
    </div>
</body>
```



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

3. ACCESO AL DOCUMENTO DESDE CÓDIGO

EJERCICIO: u6e2_listado:

- *Crea una página web que tenga un listado de tipo con un de muestra.*
- *Introduce un botón en la página que, cuando lo pulses, te muestre un prompt para que el usuario introduzca un texto.*
- *Una vez cerrado el prompt el valor se añadirá como un nuevo a la lista creada.*
- *Añade dos botones más con texto “Borrar primer li” y “Borrar último li” de modo que cuando pulses el primer botón borre el primer elemento de la lista y cuando pulses el último borre el último elemento de la lista.*



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

3. ACCESO AL DOCUMENTO DESDE CÓDIGO

EJERCICIO: u6e3_formulario:

- *Crea de manera dinámica (es decir, al cargarse la página) el formulario que definimos en el ejercicio u5e1_formulario.*
- *Ten en cuenta que el formulario deberá tener los atributos necesarios para que, al crearse, tenga la misma funcionalidad que el que creaste en html.*
- *No olvides añadir las etiquetas <label> a cada uno de los elementos.*



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.4. DIFERENCIAS EN LAS IMPLEMENTACIONES DEL MODELO

Adaptaciones de código para diferentes navegadores:

- Para mejorar la compatibilidad podemos crear de forma explícita las constantes predefinidas:
 - `alert(Node.DOCUMENT_NODE); // Devolvería 9`
 - `alert(Node.ELEMENT_NODE); // Devolvería 1`
 - `alert(Node.ATTRIBUTE_NODE); // Devolvería 2`



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.5. DIFERENCIAS EN LAS IMPLEMENTACIONES DEL MODELO

Adaptaciones de código para diferentes navegadores:

```
if(typeofNode == "undefined") {  
  varNode = {  
    ELEMENT_NODE: 1,  
    ATTRIBUTE_NODE: 2,  
    TEXT_NODE: 3,  
    CDATA_SECTION_NODE: 4,  
    ENTITY_REFERENCE_NODE: 5,
```

```
    ENTITY_NODE: 6,  
    PROCESSING_INSTRUCTION_NO DE: 7,  
    COMMENT_NODE: 8,  
    DOCUMENT_NODE: 9,  
    DOCUMENT_TYPE_NODE: 10,  
    DOCUMENT_FRAGMENT_NODE:  
      11,  
    NOTATION_NODE: 12  
  };  
}
```



6. UTILIZACIÓN DEL MODELO DE EVENTOS DEL DOCUMENTO (DOM)

6.5. DIFERENCIAS EN LAS IMPLEMENTACIONES DEL MODELO

EJERCICIO: u6e4_generadorformularios:

Crea una aplicación que te permitirá generar tus propios formularios de manera dinámica. Para ello dibuja una tabla de una sola fila y varias columnas. En cada columna habrá un botón que realice lo siguiente:

- *Crear un input de tipo texto. Le preguntará al usuario mediante un prompt qué nombre (atributo name) tiene el input.*
- *Crear un input de tipo password. Le preguntará al usuario mediante un prompt qué nombre (atributo name) tiene el input.*
- *Crear un textarea. Le preguntará al usuario el nombre y generará automáticamente un textarea de 40 columnas y 5 filas.*
- *Crear un label. Preguntará al usuario a qué input va referido (atributo for).*
- *Crear una imagen. Preguntará al usuario qué ruta tiene la imagen (atributo src).*
- *Crear un checkbox. Preguntará al usuario el nombre y el valor (atributos name y value).*
- *Crear un radio. Preguntará al usuario el nombre y el valor (atributos name y value).*
- *Crear un botón (submit). Preguntará al usuario el nombre y el valor (atributos name y value).*

